Part 1: Schema Design

Q1: Row Key and Sharding
I propose using City#Make#ModelYear#DOL_Vehicle_ID as the row key. I would change city and make names to uppercase so the keys stay consistent. Bigtable stores rows in lexicographic order, so records with the same city and make are stored next to each other. For example, a query for vehicles in Seattle can scan the SEATTLE# prefix, and a query for Teslas in Seattle can scan the smaller SEATTLE#TESLA# prefix. Model Year also allows range scans. For example, to find Seattle Teslas from 2020 to 2023, we can scan from SEATTLE#TESLA#2020 up to SEATTLE#TESLA#2024. The DOL Vehicle ID at the end makes each row key unique.

This key helps avoid hotspotting because it starts with City, not a value that keeps increasing, such as a timestamp or sequential ID. When new records are added, the writes go to many different cities' key ranges and tablets, instead of all going to the last tablet. However, hotspots can still happen if a few popular cities or makes get much more traffic than others.

One limitation is that a query for all Teslas in every city cannot use Make as a prefix, because City comes first. To fix this, I would create a second index table with the key Make#City#ModelYear#DOL_Vehicle_ID. However, this needs extra writes, and since Bigtable does not support transactions across rows, the two tables may be inconsistent for a short time. Adding a small hash or salt prefix could also spread the workload, but then queries would need to scan multiple prefixes and combine the results.

Q2: Column Families
I propose three column families, grouped by access pattern:

vehicle: Make, Model, Model Year, Electric Vehicle Type, Electric Range, Base MSRP
location: County, City, State, Postal Code, Vehicle Location, 2020 Census Tract, Legislative District
registration: VIN, DOL Vehicle ID, CAFV Eligibility, Electric Utility

This design improves read performance. Column families can be stored in separate locality groups and SSTables, so an application that only needs vehicle information can read just the vehicle family without reading location data. This means less disk I/O and less data sent over the network from tablet servers. Each family can also have its own settings. For example, the vehicle family is read often, so it could be kept in memory, while the location family could use stronger compression to save space.

Keeping only a few column families also makes the schema easier to manage. Each family can have its own versioning, data retention, and access settings. For example, access to the more sensitive location and registration families could be limited. Even though City, Make, and Model Year are already in the row key, I would also store them as columns so they are easy to use when processing records.

Q3: Horizontal Scaling: Hashed vs. Range-Based Sharding
A hashed shard key on VIN spreads records evenly across shards. This balances data and write traffic and avoids hotspots, so it works well for write-heavy workloads. A lookup by VIN can go directly to the one shard that has the record. However, range queries are slow. For example, a query for vehicles from 2020 to 2023 has to check every shard and combine the results, because records are placed by hashed VIN instead of Model Year. This adds more work for the whole cluster.

A range-based shard key on Model Year is better when queries by year are the main priority. For example, a query for vehicles from 2020 to 2023 only needs to go to the shards that hold those years. However, there are only a small number of distinct model years, so it is hard to split the data into many even partitions. Also, most vehicles are from recent years, so some shards get much more traffic than others. New registrations also mostly go to the newest year, which creates a hotspot and lowers cluster performance.

Overall, hashed sharding gives better load balance, while range-based sharding makes year-range queries faster. Another option is to use a hashed prefix followed by Model Year. This spreads data and writes evenly across shards and keeps records sorted by year inside each shard, but year-range queries would still need to go to all shards.

This comparison is separate from my row key in Q1. For the Bigtable schema, I prefer the City-first key because it supports fast queries by City and by City + Make, and it still spreads writes across different cities. I would use a hashed prefix instead if balancing write traffic became more important than fast prefix scans.

Part 4: Performance Analysis

I measured 500 requests to each insert endpoint with a Python script (perf_test.py). I ran it from my laptop and from the VM (localhost). I repeated each test a few times, and the results were consistent.

Test location	Endpoint	Average	Median	p95
Laptop	/insert-fast (w=1)	104.80 ms	74.08 ms	164.69 ms
Laptop	/insert-safe (w=majority)	110.99 ms	76.10 ms	160.12 ms
VM(localhost)	/insert-fast (w=1)	19.92 ms	8.07 ms	9.49 ms
VM(localhost)	/insert-safe (w=majority)	19.93 ms	9.91 ms	11.82 ms

From my laptop, the two endpoints had very similar latency. The average for /insert-safe was about 6 ms higher than /insert-fast, but the median was only about 2 ms higher. This is because most of the time was spent on the network between my laptop and the VM (about 70 ms), so the small extra cost of replication is hard to see.

From the VM, the network time was removed, so the difference was clearer. The median for /insert-safe was about 1.8 ms higher. This is because w=majority waits for 2 of 3 nodes to confirm the write, while w=1 only waits for the primary. The averages were higher than the medians because a few requests were very slow, so the median is a better comparison.

Based on the CAP theorem, w=1 gives lower latency and higher availability, but a write can be lost if the primary crashes before copying it to a secondary. I would choose w=1 for real-time telemetry from EV charging stations. Each station sends its status every few seconds, so losing a few updates is fine because the next update replaces them. Here, fast writes matter more than saving every update.

Appendix: perf_test.py

```python
import time
import requests
import statistics

def run_time(url, payload, n):
    times = []
    # Warm-up: send 5 untimed requests so one-time setup costs don't affect the
results
    for _ in range(5):
        requests.post(url=url, json=payload, timeout=10)

    for _ in range(n):
        start = time.perf_counter()
        response = requests.post(url=url, json=payload, timeout=10)
        end = time.perf_counter()
        response.raise_for_status()
        times.append(end - start)
    return times

payload = {
    "Make": "TEST",
    "Model": "LatencyTest",
    "Model Year": 2025
}
```

```
# fast_url = "http://localhost:8000/insert-fast"
# safe_url = "http://localhost:8000/insert-safe"
fast_url = "http://34.121.39.49:8000/insert-fast"
safe_url = "http://34.121.39.49:8000/insert-safe"

fast_times = run_time(fast_url, payload, 500)
safe_times = run_time(safe_url, payload, 500)

avg_fast = statistics.mean(fast_times)
avg_safe = statistics.mean(safe_times)

print(f"insert-fast avg: {avg_fast*1000:.2f} ms")
print(f"fast median: {statistics.median(fast_times)*1000:.2f} ms")
print(f"fast p95: {sorted(fast_times)[int(0.95*len(fast_times))]*1000:.2f} ms")
print(f"insert-safe avg: {avg_safe*1000:.2f} ms")
print(f"safe median: {statistics.median(safe_times)*1000:.2f} ms")
print(f"safe p95: {sorted(safe_times)[int(0.95*len(safe_times))]*1000:.2f} ms")
```